

✅ Résumé of the Mini-Project (Simple & Clear)

Your teacher gave you a 6-page mini-project with **one big objective**:

➡ **Study multiple sorting algorithms theoretically AND experimentally.**

This means:

✔ You must:

1. Write the C code of each sorting algorithm.
2. Analyze the theoretical complexity (best, worst).
3. Test them on random arrays of different sizes.
4. Measure execution time (5 times for each size).
5. Draw graphs of execution time vs input size.
6. Compare theoretical vs experimental results.
7. Write a final report + include C source code.

✅ 1. Bubble Sort (Classical & Optimized)

Idea

- Compare each pair of adjacent elements.
- If they are in the wrong order → swap.
- After each full pass, the largest element “bubbles” to the end.

Optimized Bubble Sort

- Same thing, **but stops earlier** because the last part becomes sorted automatically.
- Uses a variable `m` to reduce the range every pass.

Complexity

- **Best case:** $O(n)$ (if array already sorted, only one pass)
- **Worst case:** $O(n^2)$

✓ 2. Gnome Sort

Idea

- Look at items `i` and `i+1`.
- If they are in order → move forward.
- If not → swap and move backwards.
- Similar to a person fixing a line of pots one by one.

Complexity

- **Best case:** $O(n)$
- **Worst case:** $O(n^2)$

✓ 3. Radix Sort

Idea

- Sort integers **digit by digit** (units → tens → hundreds...)
- Uses a helper function to extract the i -th digit.
- Uses a stable linear sorting method (counting sort-like) for each digit.

Complexity

- **Best case:** $O(n \times k)$
- **Worst case:** $O(n \times k)$

Where k = number of digits.

✓ 4. Quick Sort

Idea

- Choose a pivot.
- Partition: put smaller elements to the left, larger to the right.
- Recursively sort left and right parts.
- Very fast in practice.

Complexity

- **Best case:** $O(n \log n)$
- **Average:** $O(n \log n)$
- **Worst case:** $O(n^2)$ (when array is already sorted badly chosen pivot)

✓ 5. Heap Sort

Idea

- Build a **heap** (a nearly complete binary tree).
- Repeatedly remove the smallest (or largest) element.
- Each removal fixes the heap again.
- Build-heap + repeated delete-min.

Complexity

- Building heap: $O(n)$
- Sorting: $O(n \log n)$
- **Total:** $O(n \log n)$

✓ 6. Bucket Sort

Idea

- Numbers are assumed inside $[0,1)$.
- Put each element in a “bucket” depending on its value.
- Sort each bucket (usually with insertion sort).
- Concatenate all buckets.

Complexity

- **Best case:** $O(n)$ (buckets evenly filled)
- **Worst case:** $O(n^2)$ (everything falls in one bucket)

★ SUMMARY TABLE (your professor will love this)

Algorithm	Best Case	Worst Case	Notes
Bubble Sort	$O(n)$	$O(n^2)$	Very slow
BubbleSortOpt	$O(n)$	$O(n^2)$	Slightly faster
Gnome Sort	$O(n)$	$O(n^2)$	Similar to insertion sort logic
Radix Sort	$O(nk)$	$O(nk)$	Uses digit-by-digit stable sort
Quick Sort	$O(n \log n)$	$O(n^2)$	Fastest in practice
Heap Sort	$O(n \log n)$	$O(n \log n)$	Very reliable
Bucket Sort	$O(n)$	$O(n^2)$	Only works if values $\in [0,1)$